

```
#include <stdio.h>
int main()
{
    printf("Hello, World!");
    return 0;
}
```

Reconocimiento de Elementos en el Desarrollo de un Programa Informático



ALBERTO FLORIDO RAMOS - 1ºA DAM

ÍNDICE

1. EXPLICACIÓN DE LOS CONCEPTOS BÁSICOS.	<u>3-5</u>
2. CLASIFICACIÓN DE LENGUAJES DE PROGRAMACIÓN.	<u>6</u>
3. IDENTIFICACIÓN DE PARADIGMAS DE PROGRAMACIÓN MEDIANTE EJEMPLOS	<u>7</u>
4. ACTIVIDAD COTIDIANA DE MANERA IMPERATIVA Y DECLARATIVA	<u>8-9</u>

1. EXPLICACIÓN DE LOS CONCEPTOS BÁSICOS

- Código fuente: es el código que escribimos en el lenguaje de programación que usamos para nuestro programa.
- Código objeto: cuando se completa el proceso de compilación, el resultado se refleja en este código.
- Código ejecutable: al enlazar el código objeto con las librerías necesarias, lo que sale es el código ejecutable.

Fases por las que pasa un programa

1. **Análisis léxico**: se trata de comprobar si los elementos del código son del lenguaje de programación correspondiente.
2. **Análisis sintáctico**: se debe analizar si el orden de los elementos en el código está bien.
3. **Análisis semántico**: en esta fase hay que determinar que las sentencias que ha escrito el programador tengan sentido.
4. **Generación de código intermedio**: si no hay ningún error previo, se genera una representación intermedia (fase opcional).
5. **Optimización de código intermedio**: al igual que la fase anterior, esta también es opcional. Es mejorar el código intermedio.
6. **Generación de código final**: aquí podremos generar código objeto para que reciba las instrucciones de la CPU.

Ejemplos concretos de los conceptos

Para esta línea de código *int total = "hola"*; tenemos lo siguiente:

- **Código fuente**: el código pasa por el análisis léxico y sintáctico, pero en el semántico da error porque no tiene sentido, lo puedes arreglar poniendo *int total = 5 + 3*.
- **Código objeto**: este código es compilado y se convierte en un archivo binario intermedio.
- **Código ejecutable**: este de aquí participa en la última fase, en conjunto con el *enlazador (linker)*.

2. CLASIFICACIÓN DE LENGUAJES DE PROGRAMACIÓN

NIVEL DE ABSTRACCIÓN:

- Alto: el programador puede leer y escribir código de forma sencilla, sin atender a muchos detalles. Ejemplo: Python y Java son lenguajes de alto nivel, pues para declarar una variable, sus elementos son *nombre = "String"*, y eso se entiende (en caso de Java, hay que especificar el tipo de variable, pero sigue siendo de alto nivel).
- Medio: este tipo de lenguajes se va acercando a las características del hardware. Para cada elemento hay que especificar dichas características, no es tan intuitivo. Ejemplos: C o Fortran. Aunque C no sea complicado, su arquitectura se acerca al hardware, y te permite interactuar con ello con más libertad.
- Bajo: aquí, los lenguajes son muy complicados de entender, difícil trabajar con ellos, aunque muy simples para trabajar, ya que no necesitan de un compilador ni intérprete porque operan muy cerca del hardware. Ejemplos: el lenguaje máquina y los ensambladores trabajan casi directamente con el hardware del ordenador.

En cuanto a los paradigmas del programación,
distinguimos dos:

- Imperativos: aquellos que, mediante un conjunto de instrucciones y expresiones, permiten crear un algoritmo que lleve a un resultado.
- Declarativos: estos solo te dan los valores que consiguen llevar al final del resultado, sin decir explícitamente qué se ha hecho para llegar a dicho resultado.

3. IDENTIFICACIÓN DE LOS PARADIGMAS DE PROGRAMACIÓN MEDIANTE EJEMPLOS

Fragmento 1: imperativo. El programador crea la sintaxis del código y el programa ejecuta línea por línea de forma explícita, dando lugar al resultado.

Fragmento 2: declarativo. Esto es debido a que, en la consulta, el usuario pide los nombres de los empleados, y éste solamente recibe los nombres. No obstante, no recibe información sobre el algoritmo que se ha creado internamente.

Fragmento 3: imperativo. El programa calcula internamente el factorial, definiendo previamente las propiedades de los números factoriales. A la hora de imprimir el resultado, solamente da el resultado de todas esas operaciones, pero no te especifica el procedimiento.

Fragmento 1: declarativo. Al comprobar cada producto uno por uno, te explica por qué hay productos que no están filtrados, mientras que otros sí lo están.

4. ACTIVIDAD COTIDIANA DE MANERA IMPERATIVA Y DECLARATIVA

La actividad es: estar en forma.

Imperativa:

1. No estás en forma
2. Ir al gimnasio
3. Hacer pesas y algo de cardio
4. Comer sano y hacer buena dieta
5. Dormir bien
6. Repetir durante meses
7. Resultado: estás en forma

Declarativa:

Procedimiento

No estás en forma ...



... Estás en forma

VENTAJAS

- Imperativa: es más claro y directo. Facilita un procedimiento para poder llegar al resultado.
- Declarativa: es más sencillo y conciso, ya que solo indica el estado inicial y final. Permite centrarse en el objetivo, sin detallar las acciones intermedias.

DESVENTAJAS

- Imperativa: te hace seguir un procedimiento sin explicar el motivo del porqué, y puede resultar largo y repetitivo a la larga.
- Declarativa: no explica cómo llegar al resultado, y proporciona menos información práctica.

Hecho por: Alberto Florido Ramos

Correo: albertofloram05@gmail.com

1ºA DAM

MUCHAS GRACIAS POR VER